Technische Hochschule Augsburg

Fakultät für Informatik

Thesis

zur Erlangung des Grades
Bachelor of Engineering in Mathematik

Proximitätssuche von EPLAN-Makros auf Basis des LambdaRank-Algorithmus

erstellt von Andreas Alstetter

Andreas Alstetter

Pfarrer-Kempter Str. 4
89358 Kammeltal/Behlingen
T +49 3141-5926
andreas.alstetter@tha.de

Matrikelnummer:
2147455

Technische Hochschule Augsburg

An der Hochschule 1
D-86161 Augsburg
T +49 821 5586-0
F +49 821 5586-3222
www.tha.de
info@tha.de

Erstprüfer	Prof. Dr. Thomas Kirchmeier
Zweitprüfer	Prof. Dr. Jürgen Grinninger
Themenvergabe am	7. September 2026
Eingereicht am	7. September 2026
Geheimhaltungsvereinbarung	Nein

Inhaltsverzeichnis

1 Motivation

Die zunehmende Digitalisierung industrieller Prozesse führt zu einer stetig wachsenden Menge an Prozess-, Produkt- und Engineeringdaten. Diese Informationen liegen sowohl in strukturierter Form, beispielsweise als technische Merkmale und Artikeldaten, als auch in unstrukturierter Form wie Datenblätter, Produktbeschreibungen oder Spezifikationen vor. Gleichzeitig steigt die Bedeutung datengetriebener Methoden für die Innovations- und Wettbewerbsfähigkeit industrieller Unternehmen. Methoden der Künstlichen Intelligenz bieten hierbei die Möglichkeit, vorhandene Datenbestände systematisch auszuwerten und für die Unterstützung technischer Entscheidungsprozesse nutzbar zu machen (Bundesministerium für Wirtschaft und Energie 2026).

Insbesondere im Anlagen- und Maschinenbau stellt die hohe Variantenvielfalt technischer Komponenten eine besondere Herausforderung dar. Unternehmen verwalten häufig umfangreiche Artikelbestände mit einer Vielzahl technischer Merkmale, während gleichzeitig große Mengen an Dokumentationen und Datenblättern verfügbar sind. Die Auswahl geeigneter Komponenten sowie die Zuordnung zu bereits existierenden technischen Lösungen erfolgt dabei oftmals auf Basis manueller Recherche und der Erfahrung einzelner Mitarbeiter. Dies führt zu erhöhtem Bearbeitungsaufwand, eingeschränkter Skalierbarkeit sowie einer starken Abhängigkeit von domänenspezifischem Expertenwissen (Pfenning 2022).

Aktuelle wissenschaftliche Arbeiten zeigen, dass Verfahren der Künstlichen Intelligenz erfolgreich zur Wissensextraktion, Automatisierung und Entscheidungsunterstützung in industriellen Anwendungen eingesetzt werden. Ein zentraler Aspekt besteht dabei in der Verarbeitung heterogener Datenquellen sowie der Nutzung der daraus gewonnenen Informationen für technische Entscheidungsprozesse. Für solche Aufgaben haben sich Empfehlungssysteme als geeigneter Ansatz etabliert. Sie unterstützen Anwender dabei, aus einer Vielzahl möglicher Alternativen die für einen konkreten Anwendungsfall am besten geeignete Option auszuwählen. Ursprünglich vor allem im E-Commerce eingesetzt, finden Empfehlungssysteme zunehmend Anwendung in industriellen Domänen, beispielsweise bei der Produktauswahl, Wiederverwendung bestehender Lösungen oder bei Konfigurationsaufgaben (Joshi u. a. 2024).

Eine im industriellen Engineering weit verbreitete Softwareplattform ist EPLAN. Sie unterstützt Ingenieure bei der Planung, Dokumentation und Standardisierung elektrotechnischer Systeme. Neben Schaltplänen und Stücklisten ermöglicht EPLAN die Nutzung wiederverwendbarer Schaltplanfragmente, die als *Fenstermakros* bezeichnet werden (EPLAN GmbH & Co. KG 2025). Darüber hinaus stellt die EPLAN-Artikelverwaltung strukturierte Informationen zu elektrotechnischen

und mechanischen Komponenten bereit (EPLAN GmbH & Co. KG 2026b). Die dort hinterlegten Merkmale bilden eine wertvolle Informationsquelle für wissensbasierte und datengetriebene Anwendungen.

Aus dieser Ausgangssituation ergibt sich das Potenzial für ein KI-basiertes Empfehlungssystem zur automatisierten Zuordnung von EPLAN-Artikeln (EPLAN GmbH & Co. KG 2026b) zu EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025). Die hierfür relevanten Informationen sind bereits in den vorhandenen Artikelstammdaten und technischen Beschreibungen enthalten, werden jedoch bislang nur begrenzt für die automatisierte Entscheidungsunterstützung genutzt. Durch die strukturierte Auswertung dieser Informationen können Zusammenhänge zwischen Artikeln und Makros identifiziert und für die automatisierte Empfehlung geeigneter Fenstermakros verwendet werden.

Im Rahmen dieser Arbeit wird hierfür ein datengetriebenes Learning-to-Rank-System entwickelt, das für einen gegebenen Artikel eine Rangfolge potenziell geeigneter Fenstermakros erzeugt. Ziel ist es, die Wahrscheinlichkeit zu maximieren, das korrekte Makro innerhalb der obersten Rangpositionen bereitzustellen. Dadurch kann der manuelle Such- und Zuordnungsaufwand reduziert, vorhandenes Unternehmenswissen systematisch nutzbar gemacht und die Konsistenz technischer Entscheidungen verbessert werden. Das Projekt leistet somit einen Beitrag zur praktischen Umsetzung datengetriebener und KI-gestützter Entscheidungsunterstützung im Industrie-4.0-Umfeld (Nayak 2025).

Abbildung 1.1 veranschaulicht die grundlegende Idee der automatisierten Zuordnung von EPLAN-Artikeln zu EPLAN-Fenstermakros.

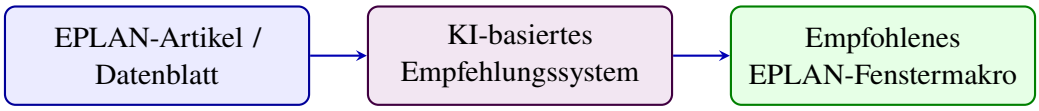


Abbildung 1.1: Grundidee der automatisierten Zuordnung von EPLAN-Artikeln zu EPLAN-Fenstermakros.

2 Stand der Technik

2.1 Learning-to-Rank in industriellen Empfehlungssystemen

Rankingprobleme treten in einer Vielzahl von Anwendungsgebieten auf, beispielsweise in Suchmaschinen, Produktempfehlungssystemen oder technischen Informationssystemen. Ziel ist es, für eine gegebene Anfrage die verfügbaren Kandidaten im Bezug auf ihre Relevanz zu bewerten und anschließend in eine Rangfolge zu bringen (Liu 2009).

Im industriellen Umfeld gewinnen Rankingverfahren zunehmend an Bedeutung, da häufig eine große Anzahl technischer Komponenten, Dokumente oder wiederverwendbarer Engineering-Artefakte zur Verfügung steht. Anstelle einer Entscheidung zwischen passend und nicht passend ermöglichen Rankingmodelle eine Priorisierung von möglichen Lösungen und unterstützen Anwender somit bei der Auswahl der geeignetsten Alternative. Für die Umsetzung solcher Aufgaben haben sich Learning-to-Rank-Verfahren etabliert. Diese nutzen strukturierte und beschriebene Anfrage-Kandidat-Paare. Damit werden aus historischen Zuordnungen eine Relevanzfunktion gelernt, die neue Kandidaten automatisch bewerten kann.

Im Gegensatz zu klassischen Klassifikationsverfahren wird dabei nicht die Vorhersage einer Klasse optimiert, sondern die Qualität der einer resultierenden Rangliste (Liu 2009; Burges 2010). Die vorliegende Arbeit folgt diesem Ansatz. Aus den Merkmalen eines EPLAN-Artikels (EPLAN GmbH & Co. KG 2026b) sowie den Eigenschaften potenzieller EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025) werden Anfrage-Kandidat-Paare erzeugt. Ein Learning-to-Rank-Modell bewertet diese Kandidaten im Anschluss hinsichtlich ihrer Eignung und erstellt eine Rangliste möglicher Makroempfehlungen. Abbildung 2.1 zeigt schematisch das grundlegende Vorgehen.

2.1.1 Aktuelle Anwendungsbeispiele

Learning-to-Rank-Verfahren werden heute in zahlreichen industriellen Anwendungen eingesetzt. Bekannte Einsatzgebiete umfassen Produktempfehlungen im E-Commerce, die Priorisie-



Abbildung 2.1: Grundprinzip eines Learning-to-Rank-Systems zur Makroempfehlung.

rung von Suchergebnissen in Unternehmensdatenbanken sowie technische Assistenzsysteme für Engineering- und Serviceprozesse. Im E-Commerce nutzen Unternehmen wie Amazon, Alibaba oder eBay Learning-to-Rank-Verfahren zur Sortierung von Produktsuchergebnissen und Produktempfehlungen. Dabei werden strukturierte Produktmerkmale, Nutzerinteraktionen und historische Kaufdaten kombiniert, um die Relevanz einzelner Produkte innerhalb einer Ergebnisliste zu bestimmen (Kabir u. a. 2024).

2.2 Bewertungsmetriken für Rankingverfahren

Die Bewertung von Rankingverfahren unterscheidet sich bis auf Recall@k von der Bewertung klassischer Klassifikations- oder Regressionsverfahren. Während bei einer Klassifikation die korrekte Zuordnung einzelner Objekte im Vordergrund steht, wird bei Rankingproblemen die Qualität der gesamten Ergebnisreihenfolge bewertet. Hierfür haben sich spezielle Rankingmetriken etabliert, welche unterschiedliche Aspekte einer Rangliste erfassen (Liu 2009). Für Learning-to-Rank-Systeme werden insbesondere Recall@k, Mean Reciprocal Rank (MRR) sowie Normalized Discounted Cumulative Gain (nDCG) eingesetzt. Diese Metriken ermöglichen die Bewertung der Position relevanter Ergebnisse sowie der Gesamtqualität einer Rangliste.

2.2.1 Recall@k

Recall@k gehört zu den grundlegenden Bewertungsmetriken des Information Retrievals und beschreibt den Anteil der Anfragen, bei denen mindestens ein relevantes Ergebnis innerhalb der ersten k Rangpositionen gefunden wird (Manning u. a. 2008). Diese Metrik berechnet sich wie folgt:

$$Recall@k = \frac{\text{\#Anfragen mit relevantem Ergebnis in Top-}k}{\text{\#Anfragen}} \tag{2.1}$$

mit

k Anzahl der betrachteten Rangpositionen
#Anfragen mit relevantem Ergebnis in Top-k Anzahl erfolgreicher Anfragen
#Anfragen Gesamtzahl aller Anfragen

Der Wertebereich liegt zwischen 0 und 1. Ein Wert von 1 bedeutet, dass sich für jede Anfrage mindestens ein relevantes Ergebnis innerhalb der Top-k Positionen der Rangliste befindet. In Ranking- und Empfehlungssystemen wird Recall@k häufig verwendet, um die praktische Verwendung eines Systems zu beurteilen zu können. Der Anwender betrachtet dabei in der Regel nur die ersten Ergebnisse einer Rangliste (Manning u. a. 2008).

2.2.2 Mean Reciprocal Rank

Der Mean Reciprocal Rank (MRR) zählt neben Recall@k zu etablierten Bewertungsmetriken des Information Retrievals und wird sowohl in klassischen Suchsystemen als auch in modernen Rankingverfahren eingesetzt (Voorhees und Tice 1999; Zhu u. a. 2024). Mit dieser Kennzahl wird die Position des ersten relevanten Ergebnisses innerhalb einer Rangliste bewertet.

Die Berechnung ergibt sich wie folgt:

$$MRR = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{rank_i} \tag{2.2}$$

mit

N Anzahl der ausgewerteten Anfragen
$rank_i$ Rangposition des ersten relevanten Ergebnisses für Anfrage i

Der Wertebereich liegt zwischen 0 und 1. Ein hoher Wert zeigt an, dass ein relevantes Ergebnis bereits an einer der vorderen Positionen einer Rangliste erscheint. Da frühe Treffer stärker gewichtet werden als spätere Treffer, eignet sich die Kennzahl besonders zur Bewertung der schnellen Auffindbarkeit relevanter Ergebnisse.

2.2.3 Normalized Discounted Cumulative Gain

Der Discounted Cumulative Gain (DCG) wurde von Järvelin und Kekäläinen zur Bewertung verschiedener Relevanzstufen von Suchergebnissen eingeführt und zählt zu den wichtigsten Bewertungsmetriken für Ranking- und Suchsysteme (Järvelin und Kekäläinen 2002). Die Kennzahl berücksichtigt sowohl die Relevanz eines Ergebnisses als auch die Position innerhalb der Rangliste. Relevanter Ergebnisse auf frühen Rangpositionen leisten dadurch einen höheren Beitrag zur Gesamtbewertung.

Die Berechnung des DCG erfolgt über

$$DCG@k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)} \tag{2.3}$$

mit

k Anzahl der betrachteten Rangpositionen
i Rangposition innerhalb der Rangliste

rel_i Relevanzwert des Ergebnisses auf Rangposition i

Um eine bessere Vergleichbarkeit zu schaffen, findet eine Normierung auf die ideale Rangliste statt. Hieraus ergibt sich der Normalized Discounted Cumulative Gain (nDCG):

$$nDCG@k = \frac{DCG@k}{IDCG@k} \tag{2.4}$$

mit

$DCG@k$ DCG-Wert der betrachteten Rangliste
$IDCG@k$ DCG-Wert der idealen Rangliste

Der Wertebereich liegt zwischen 0 und 1. Ein Wert von 1 entspricht der optimal möglichen Rangfolge.

2.2.4 Hierarchische Bewertungsmetriken

Nicht jede fachliche Fehlentscheidung hat in technischen Empfehlungssystemen dieselbe Bedeutung. Ein Ergebnis kann beispielsweise ein Teil einer technisch verwandten Produkt- oder Makrofamilie angehören, aber trotz fehlender exakter Übereinstimmung für den Anwender weiterhin von Interesse sein. Zur Berücksichtigung solcher Beziehungen werden hierarchische Bewertungsmetriken eingesetzt.

Diese basieren typischerweise auf dem Discounted Cumulative Gain (Järvelin und Kekäläinen 2002) und verwenden unterschiedliche Relevanzen in abgestufter Reihenfolge.

Eine mögliche Relevanzfunktion lautet:

$$rel = \begin{cases} 3, & \text{exakte Übereinstimmung} \\ 2, & \text{gleiche Familie} \\ 0, & \text{keine fachliche Beziehung} \end{cases} \tag{2.5}$$

Dadurch wird eine Rangliste bevorzugt, welche fachlich ähnliche Kandidaten auf höheren Positionen platziert. Hierarchische Bewertungsmetriken eignen sich insbesondere für industrielle Empfehlungssysteme, bei denen zwischen exakter Übereinstimmung und fachlicher Nähe unterschieden werden muss.

2.3 Learning-to-Rank mit LambdaRank

2.3.1 Grundprinzip von Learning-to-Rank

Learning-to-Rank (LTR) bezeichnet eine Klasse an maschinellen Lernverfahren, die das Ziel besitzen die Reihenfolge in einer Ergebnisliste zu optimieren (Liu 2009). Im Gegensatz zu klassischen Klassifikations- oder Regressionsansätzen, bewertet Learning-to-Rank keine einzelnen Objekte isoliert, sondern die relative Relevanz dieser Objekte in einer Kandidatenmenge. Die Aufgabe eines Rankingmodells besteht darin, dass für eine Anfrage die relevanten Dokumente anhand ihrer Merkmale bewertet werden und anschließend in eine Rangliste überführt werden.

2.3.2 LambdaRank

LambdaRank ist eines der bekanntesten Verfahren im Bereich des Learning-to-Rank. (Burges 2010)

Die Grundidee von LambdaRank ist der paarweise Vergleich von Kandidaten. Dabei wird für zwei Kandidaten i und j mit den vorhergesagten Ergebnissen s_i und s_j die Wahrscheinlichkeit berechnet, dass Kandidat i vor Kandidat j eingeordnet werden sollte:

$$P_{ij} = \frac{1}{1 + e^{-\sigma(s_i - s_j)}} \tag{2.6}$$

mit

P_{ij} Wahrscheinlichkeit, dass Kandidat i vor Kandidat j gerankt wird
s_i vorhergesagter Score von Kandidat i
s_j vorhergesagter Score von Kandidat j
σ Skalierungsparameter der Sigmoidfunktion

Die Optimierung erfolgt anschließend durch die Änderung einer Rankingmetrik. Typischerweise wird dabei die Normalized Discounted Cumulative Gain (nDCG)-Funktion verwendet. Dabei werden die Gradienten anhand ihres Einflusses auf die Qualität der Rangliste gewichtet.

Der sogenannte Lambda-Gradient berechnet sich vereinfacht zu:

$$\lambda_{ij} = \frac{\partial L_{ij}}{\partial s_i} \cdot |\Delta nDCG| \tag{2.7}$$

mit

λ_{ij} gewichteter Gradient des Kandidatenpaares
$\frac{\partial L_{ij}}{\partial s_i}$ Ableitung der Verlustfunktion
$|\Delta nDCG|$ Änderung der Rankingqualität bei Vertauschung der Kandidaten

Kandidatenpaare, deren Reihenfolge einen größeren Einfluss auf die Ranglisten-Qualität besitzen, erhalten einen höheren Einfluss auf den Lernprozess. Dadurch wird das Modell auf die Qualität der Ergebnisliste optimiert und nicht auf die Vorhersage korrekter Einzelentscheidungen (Burges 2010).

2.3.3 LambdaMART und LightGBM

LambdaRank wird in modernen Rankinganwendungen häufig mit Gradient Boosted Decision Trees kombiniert, was als LambdaMART bezeichnet wird. Diese zählt zu den erfolgreichsten Verfahren im Bereich der strukturierten Rankingdaten (Burges 2010).

Die zugrunde liegende Logik ist eine additive Zusammensetzung von Entscheidungsbäumen:

$$F(x) = \sum_{m=1}^{M} \gamma_m h_m(x) \tag{2.8}$$

mit

$F(x)$ vorhergesagter Gesamtscore
$h_m(x)$ Entscheidungsbaum m
γ_m Gewichtung des Baumes
M Anzahl aller Entscheidungsbäume

Dabei versucht jeder neu hinzugefügte Entscheidungsbaum, die Fehler des bisherigen Modells zu korrigieren.

Eine weit verbreitete Implementierung in der Industrie ist der `LGBMRanker` der LightGBM-Bibliothek. Dabei wird das Modell gruppenbasiert trainiert und alle Kandidaten einer Anfrage bilden eine Rankinggruppe, mit dem Ziel, die Reihenfolge der Kandidaten innerhalb einer Gruppe zu optimieren.

2.3.4 Aktuelle Bedeutung von LambdaMART

Trotz der zunehmenden Forschung im Bereich neuronaler Rankingverfahren, gehören LambdaMART-basierte Modelle weiterhin zu den leistungsfähigsten Verfahren im Bereich der strukturierten

tabellarischen Daten. Eine aktuelle Vergleichsstudie zeigt, dass Gradient-Boosted-Decision-Tree-Verfahren vor allem bei heterogenen Datensätzen noch immer State of the Art sind (Lyzhin u. a. 2023).

Lyzhin et al. vergleichen in ihrer Studie mehrere moderne Learning-to-Rank-Verfahren miteinander und kommen zu dem Ergebnis, dass LambdaMART-basierte Modelle immer noch äußerst konkurrenzfähig sind. Neue Optimierungsverfahren wie YetiRank und StochasticRank liefern dabei nur begrenzte Verbesserungen gegenüber LambdaMART-Modellen (Lyzhin u. a. 2023).

3 Zielsetzung und Vorgehensweise

3.1 Zielsetzung

Das Projektziel ist die Entwicklung eines Systems, welches automatisiert EPLAN-Artikel (EPLAN GmbH & Co. KG 2026b) zu EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025) zuordnet. Für einen vorhanden technischen EPLAN-Artikel (EPLAN GmbH & Co. KG 2026b) soll anhand einer technischen Anfrage (zum Beispiel in Form eines Datenblatts) das fachlich passendste EPLAN-Fenstermakro (EPLAN GmbH & Co. KG 2025) identifiziert werden.

Diese Problemstellung kann als Rankingaufgabe formuliert werden, bei der für jede Anfrage eine Menge bekannter EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025) vorliegt. Dabei sind alle Kandidaten dieser Menge potenzielle Ergebnisse für den 1. Rang. Das System soll die Eigenschaften eines gegebenen EPLAN-Artikels (EPLAN GmbH & Co. KG 2026b) analysieren und diese mit allen verfügbaren Makro-Kandidatenprofilen vergleichen. Anschließend sollen die Kandidaten entsprechend ihrer Relevanz geordnet werden. Ziel dabei ist es, dass das passendste EPLAN-Fenstermakro (EPLAN GmbH & Co. KG 2025) möglichst auf Rang 1 erscheint. Die weniger ähnlichen Kandidaten sollen entsprechend der Bewertung hierarchisch auf den nachfolgenden Rängen einsortiert werden.

Im Gegensatz zu Klassifikationsaufgaben liegt der Fokus dieses Systems nicht direkt auf der Bestimmung einer einzelnen Klasse oder Produktfamilie. Vielmehr geht es darum eine möglichst präzise Variantenauflösung zu erreichen. Dafür sollen die technischen Eigenschaften und Merkmale aus einer Anfrage systematisch erfasst und mit den Eigenschaften der EPLAN-Artikel (EPLAN GmbH & Co. KG 2026b) abgeglichen werden.

Die Herausforderung besteht darin, dass sich viele Produkte lediglich in einzelnen technischen Eigenschaften und Konfigurationsmerkmalen unterscheiden. Geringe Abweichungen können deshalb zu unterschiedlichen Makrozuordnungen führen. Das System muss daher in der Lage sein allgemeine technischen Zusammenhänge als auch merkmalspezifische Unterschiede zwischen ähnlichen Varianten zu erkennen.

Zur Lösung dieses Problems wird ein Learning-to-Rank-System entwickelt, welches strukturierte technische sowie lexikalische Merkmale aus Artikeldaten kombiniert, um die Relevanz von potenziellen EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025) zu bewerten. Das Modell erzeugt für jeden verfügbaren Makrokandidaten eine Relevanzbewertung und erstellt daraus eine Rangliste möglicher Empfehlungen. Ziel ist es, das korrekte Zielmakro möglichst auf den vorderen Rangpositionen, idealerweise auf Rang 1, zu platzieren.

3.2 Methodisches Vorgehen

Das methodische Vorgehen orientiert sich an der *Design Science Research Methodology* (DSRM) nach Peffers et al. (2007). Die DSRM beschreibt einen strukturierten Forschungsprozess zur Entwicklung und Evaluation technischer Artefakte und eignet sich insbesondere für die Entwicklung von IT-Systemen. Das Ziel der Methodik besteht darin, eine konkrete Problemstellung systematisch zu analysieren, ein geeignetes Artefakt zu entwickeln und den Nutzen durch eine Evaluation nachzuweisen.

Der von Peffers et al. beschriebene Forschungsprozess umfasst die folgenden Schritte:

1. Problemidentifikation und Motivation
2. Definition der Lösungsziele
3. Design und Entwicklung des Artefakts
4. Demonstration der entwickelten Lösung
5. Evaluation des Artefakts
6. Kommunikation der Ergebnisse

In dieser Arbeit bildet die DSRM den übergeordneten Rahmen. Die iterative Entwicklung des Ranking-Systems erfolgt innerhalb der Design-, Entwicklungs- und Evaluationsphase durch wiederkehrende Entwicklungszyklen.

3.3 Iterativer Entwicklungsprozess

Zur Entwicklung des Artefakts wird die Design-Science-Methodik durch die Prinzipien des *Action Design Research* (ADR) nach Sein et al. verwendet (2011). ADR verbindet dabei die Entwicklung eines Artefakts mit der fortlaufenden Anwendung, Evaluation und Verbesserung. Während die DSRM den strukturellen Prozess beschreibt, liefert ADR den Entwicklungszyklus zur kontinuierlichen Verbesserung des Systems.

Der iterative Entwicklungsprozess besteht dabei aus den folgenden Schritten:

1. Analyse des aktuellen Zustands und Identifikation von Verbesserungspotentialen
2. Recherche wissenschaftlicher und technischer Lösungen
3. Konzeption neuer Komponenten, Merkmale oder Optimierungen
4. Implementierung und Anpassung der entwickelten Ansätze
5. Durchführung von Tests und Validierungen
6. Evaluation anhand definierter Qualitätsmetriken
7. Interpretation der Ergebnisse
8. Ableitung weiterer Verbesserungen

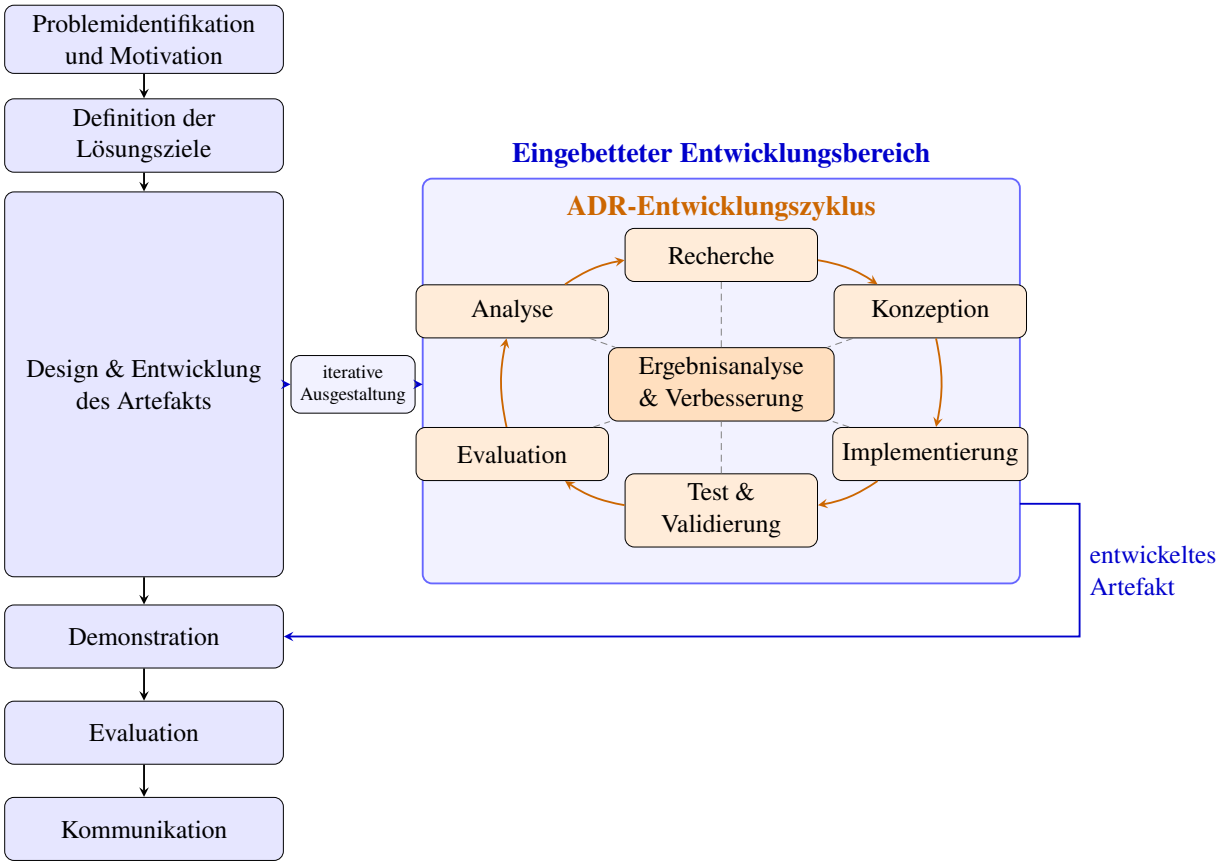


Abbildung 3.1: Verzahnung von DSRM und ADR. Die DSRM bildet den übergeordneten Forschungsrahmen, während ADR als iterativer Entwicklungszyklus innerhalb der Phase „Design & Entwicklung des Artefakts“ eingesetzt wird.

Die Ergebnisse einer Iteration bilden jeweils die Grundlage für die nachfolgenden Iterationen. Auf diese Weise entsteht ein kontinuierlicher Verbesserungsprozess, in dem Forschung, Entwicklung, Evaluation und Optimierung miteinander verzahnt sind. Die Kombination aus DSRM und ADR ermöglicht somit auf der einen Seite einen strukturierten Forschungsprozess und auf der anderen Seite eine iterative Verbesserung des entwickelten Ranking-Systems. Abbildung 3.1 veranschaulicht diese Verzahnung der beiden Methodiken.

Die theoretischen Grundlagen der verwendeten Bewertungsmetriken wurden bereits in Kapitel 2 vorgestellt. Die Anwendung dieser Metriken zur Bewertung des entwickelten Systems erfolgt in Kapitel **??**. Eine Darstellung der finalen Systemarchitektur und der vollständigen Pipeline befindet sich ebenfalls in diesem Kapitel.

4 Architekturübersicht

4.1 Gesamtarchitektur

Die entwickelte Lösung basiert auf einer Learning-to-Rank-Architektur zur automatisierten Zuordnung von EPLAN-Artikeln (EPLAN GmbH & Co. KG 2026b) zu EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025). Ausgangspunkt der Pipeline sind strukturierte Artikeldaten, technische Merkmale sowie Informationen aus den zugehörigen Makros. Nach der Datenaufbereitung werden die verfügbaren Artikel in Trainings-, Validierungs- und Testdaten aufgeteilt. Anschließend werden die Makro-Kandidatenprofile ausschließlich aus dem Trainingssplit erzeugt. Für jede Anfrage werden im Anschluss Anfrage-Kandidat-Paare gebildet. Dabei wird jeder Kandidat durch ein Makroprofil repräsentiert und mit der Anfrage verglichen. Aus diesen Paaren werden strukturierte und lexikalische Merkmale berechnet, welche als Eingabe in das spätere Learning-to-Rank-Modells dienen. Das Rankingmodell basiert auf LightGBM LambdaRank und bewertet sämtliche verfügbaren Kandidatenprofile. Aus den Relevanzwerten der Kandidaten wird anschließend eine Rangliste möglicher Makroempfehlungen gebildet. Die Pipeline umfasst neben dem eigentlichen Training auch die automatische Evaluation sowie die Erzeugung von Modellartefakten zur wiederverwendbaren Nutzung eines Modells. Abbildung 4.1 zeigt die implementierte End-to-End-Architektur des Systems.

Die Orchestrierung aller Verarbeitungsschritte erfolgt zentral über die Trainingspipeline (Training and ranking pipeline 2026).

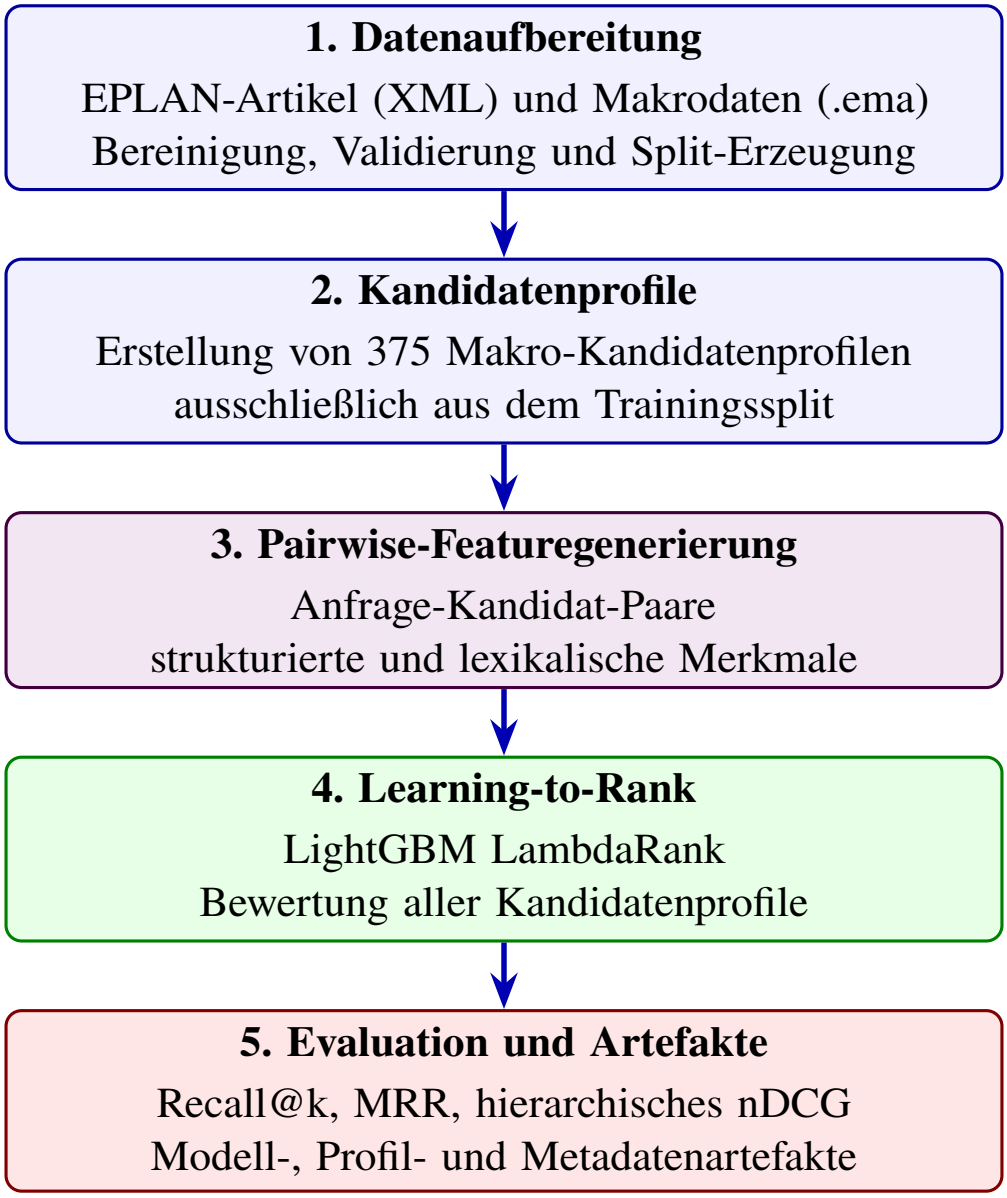


Abbildung 4.1: End-to-End-Architektur der retrievalfreien Learning-to-Rank-Pipeline.

5 Analyse der Datenstrukturen und Ableitung eines Trainingsformats

5.1 Analyse der Ausgangsdaten

Die Datengrundlage für das entwickelte System bilden EPLAN-Artikel (EPLAN GmbH & Co. KG 2026b) und EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025). Diese liegen in Form von strukturierten XML-Dateien und .ema-Dateien, einem XML-ähnlichen Format, vor (Pipeline step 911 2026). Um diese Daten für ein Learning-to-Rank-System nutzbar zu machen, müssen diese zunächst aufbereitet und in ein maschinenverständliches Format überführt werden.

Das Ziel der Datenaufbereitung besteht darin, alle technisch wichtigen Informationen der EPLAN-Artikel (EPLAN GmbH & Co. KG 2026b) sowie die strukturellen Merkmale der EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025) zu extrahieren. Auf dieser Grundlage kann die Eignung eines EPLAN-Artikels (EPLAN GmbH & Co. KG 2026b) für ein EPLAN-Fenstermakro (EPLAN GmbH & Co. KG 2025) bewertet werden. Die Umsetzung dieser Datenaufbereitung erfolgt innerhalb der entwickelten Trainings- und Rankingpipeline (Pipeline step 911 2026; Training and ranking pipeline 2026).

Zum Zeitpunkt der Durchführung dieser Arbeit umfasst der Datensatz insgesamt 1.559 EPLAN-Artikel und 375 Zielmakros, die zu 55 Makrofamilien gehören.

Tabelle 5.1: Statistische Kennzahlen des verwendeten Datensatzes

Kennzahl	**Wert**
Artikel	1.559
Makro-Targets	375
Makrofamilien	55
Singleton-Targets	213
Targets mit Support < 3	279

Tabelle 5.2: Analysierte Merkmalsgruppen der EPLAN-Fenstermakros

Merkmalsgruppe	**Extrahierte Merkmale und Bedeutung**
Function-Template-Struktur	**Merkmale:** Function-Template-Kategorien, Function-Template-Gruppen, Function-Template-IDs, Function-Template-Sequenzen, Struktur-Hashes, Sequenz-Hashes. **Bedeutung:** Function-Templates sind grundlegende Bausteine, welche die Funktionalität und Logik der EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025) im EPLAN-System (EPLAN GmbH & Co. KG 2026a) bestimmen. Die Merkmale beschreiben die funktionale Struktur und ermöglichen die Identifikation ähnlicher oder identischer Makrostrukturen.
Anschluss- und Signalstruktur	**Merkmale:** Anzahl der Anschlüsse, Anschlussrollen, Anschlussarten, Signalanzahlen, Kanalzahlen, I/O-Schemata, Klemmeninformationen. **Bedeutung:** Beschreibt die technische Funktion sowie die Anschlusslogik eines EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025).
Geräteeigenschaften	**Merkmale:** Bus-Systeme, Steckersysteme, Safety-Eigenschaften, LED-Eigenschaften, Leistungsmerkmale, Kommunikationsinformationen. **Bedeutung:** Ordnet Makros technisch im Maschinen- und Anlagenbau-Kontext ein.
Varianten-informationen	**Merkmale:** Variantenkennungen, Baugruppeninformationen, Konfigurationsmerkmale, produktspezifische Strukturinformationen. **Bedeutung:** Unterstützt die Auflösung fachlich ähnlicher Varianten, die sich nur durch einzelne technische Eigenschaften oder Konfigurationsmerkmale unterscheiden.

5.1.1 Datensatzstatistik

5.2 Analyse der Makrostrukturen

Die Analyse der Makrostrukturen der EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025) konzentriert sich auf die enthaltenen Funktions- und Strukturmerkmale, welche die Grundstruktur und Funktionalität der Makros bestimmen (Pipeline step 911 2026). Tabelle 5.2 fasst die analysierten Merkmalsgruppen, die jeweils extrahierten Informationen und deren Bedeutung zusammen.

Tabelle 5.3: Aufbereitete Merkmalsgruppen der EPLAN-Artikeldaten

Merkmalsgruppe	**Merkmale**
Identifikationsmerkmale	Hersteller, Produktfamilie, Produkttyp, Bestellnummer, Produktcodes
Elektrische Eigenschaften	Spannungswerte, Stromwerte, Leistungswerte, Frequenzangaben
Mechanische Eigenschaften	Abmessungen, Gewicht, Baugrößen, Gehäusemerkmale
Technologische Eigenschaften	Schutzarten, Kommunikationsstandards, Anschlussinformationen, spezielle Gerätefunktionen

Die Makrostrukturmerkmale werden beim Aufbau der Makrokandidatenprofile gespeichert. In der regulären Query-Kandidat-Feature-Berechnung werden diese jedoch nicht direkt mit einer unbekannten Artikelanfrage verglichen, da deren Zielmakrostruktur zur Anfragezeit nicht verfügbar ist.

5.3 Aufbereitung der Artikeldaten

Neben den vorhandenen EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025) werden auch die EPLAN-Artikeldaten (EPLAN GmbH & Co. KG 2026b) aufbereitet und analysiert (Pipeline step 911 2026). Die verwendeten Merkmalsgruppen sind in Tabelle 5.3 zusammengefasst.

Vor der Extraktion der Merkmale werden die Ursprungsdaten normalisiert und in ein einheitliches Format überführt. Dabei werden Texte, Schreibvarianten sowie technische Einheiten per Regular Expressions (RegEx) aufbereitet (Pipeline step 911 2026).

5.4 Erstellung von Makrokandidatenprofilen

Im nächsten Schritt wird für jedes bekannte EPLAN-Fenstermakro (EPLAN GmbH & Co. KG 2025) ein Makrokandidatenprofil erstellt. Ein Makrokandidatenprofil fasst dabei die technischen Eigenschaften des jeweiligen Makros zusammen. Für Merkmals-Kategorien werden die häufigsten Ausprägungen gespeichert. Für numerische Merkmale werden statistische Werte wie Mittelwert, Median, Minimum, Maximum und Häufigkeiten berechnet. Die Berechnung dieser Kandidatenprofile erfolgt innerhalb der Trainingspipeline (Pipeline step 931 2026)

Im aktuellen Datensatz entstehen dadurch 375 Makro-Kandidatenprofile. Die Erstellung der Profile erfolgt ausschließlich auf der Grundlage des Trainingssplits. Informationen aus den Validierungs- oder Testdaten werden nicht zur Profilerstellung verwendet. Für jedes Profil werden unter anderem das Zielmakro, Makrofamilie, Supportinformationen, technische Vektorrepräsentationen,

lexikalische Merkmale, Strukturmerkmale sowie statistische Eigenschaften gespeichert. Die Profilbreite ist abhängig von der untersuchten Konfiguration. Für den später als Referenz verwendeten A08-Modellstand beträgt die Profilbreite 16 Artikel pro Kandidatenprofil.

5.5 Transformation in ein Rankingproblem

Die Zuordnung der EPLAN-Fenstermakros (EPLAN GmbH & Co. KG 2025) wird nicht als Klassifikationsproblem, sondern als Ranking-Aufgabe betrachtet. Für jede Anfrage werden alle EPLAN-Fenstermakro-Kandidaten (EPLAN GmbH & Co. KG 2025) betrachtet, deren Profile ausschließlich aus dem Trainingssplit abgeleitet wurden. Eine vorgeschaltete Reduktion auf eine Top-N-Kandidatenmenge findet im aktuellen System nicht statt, sodass alle Trainingsartikel für die Paar-Bildung verwendet werden.

$$C_q = \{c_1, c_2, \ldots, c_n\} \tag{5.1}$$

Dabei bezeichnet q die Anfrage bzw. den Eingangsartikel, C_q die Menge aller Makrokandidaten und c_i einen einzelnen Makrokandidaten. Für jede Anfrage wird anschließend jeweils ein Paar (q, c_i) gebildet. Dadurch entsteht eine Rankingstruktur, in der die Makrokandidaten gegeneinander bewertet werden.

5.6 Berechnung der Pairwise-Merkmale

Für jedes Paar von Anfrage und EPLAN-Fenstermakro-Kandidat (EPLAN GmbH & Co. KG 2025) wird ein numerischer Feature-Vektor erzeugt. Die Merkmale der Anfrage-Kandidat-Paare lassen sich in mehrere Gruppen unterteilen: mengenbasierte Ähnlichkeitsmerkmale, numerische Vergleichsmerkmale, Function-Template- und Strukturmerkmale, Hash- und Identitätsmerkmale, lexikalische Merkmale, Support- und Profilmerkmale, Varianten- und Baugruppenmerkmale sowie technische Artikelmerkmale.

Zur Bewertung von Merkmalsüberschneidungen wird unter anderem die Jaccard-Ähnlichkeit verwendet:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{5.2}$$

Dabei bezeichnet A die Merkmalsmenge der Anfrage und B die Merkmalsmenge des Makrokandidaten. Der Wertebereich liegt zwischen 0 und 1.

Tabelle 5.4: Gruppen der Pairwise-Merkmale

Merkmalsgruppe	**Funktion**
Mengen- und Ähnlichkeitsmerkmale	Jaccard-Ähnlichkeiten, Overlap-Raten und Mengenüberdeckungen technischer Merkmale
Numerische Vergleichsmerkmale	Absolute und relative Differenzen technischer Kennwerte sowie Verhältnismerkmale
Function-Template- und Strukturmerkmale	Function-Templates, I/O-Schemata, Terminalfunktionen, Sequenzen und technische Strukturmerkmale
Hash- und Identitätsmerkmale	Struktur-, Sequenz- und Rollenhashes zur Identifikation technischer Zusammenhänge
Lexikalische Merkmale	Lexical-Vector-Merkmale und technische Tokens aus Produktbezeichnungen und Eigenschaftsbeschreibungen
Support- und Profilmerkmale	Supportwerte, Singleton-Kennzeichen, Profilbreiten und statistische Eigenschaften der Kandidatenprofile
Varianten- und Baugruppenmerkmale	Variantenkennungen, Baugruppeninformationen und EMA-bezogene Strukturmerkmale
Technische Artikelmerkmale	Spannungs-, Strom-, Leistungs-, Kommunikations- und Anschlussmerkmale

Um numerische Eigenschaften zu bewerten, werden Differenzmerkmale berechnet. Die absolute Differenz ist definiert als:

$$d(x, y) = |x - y| \tag{5.3}$$

Die relative Differenz wird berechnet durch:

$$d_{rel}(x, y) = \frac{|x - y|}{\max(|x|, |y|, \varepsilon)} \tag{5.4}$$

Dabei bezeichnen x den Eigenschaftswert der Anfrage, y den Eigenschaftswert des Kandidaten und ε eine kleine Konstante zur Vermeidung von Division durch Null. Durch diese Metriken können technische Kennwerte aus unterschiedlichen Größenordnungen verglichen werden.

Die weiteren Pairwise-Merkmale sind in Tabelle 5.4 zusammengefasst. Die Berechnung der Pairwise-Merkmale erfolgt innerhalb der entwickelten Trainingspipeline (Pipeline step 931 2026).

Die aktuelle Featurematrix umfasst für die Referenzkonfiguration A08 insgesamt 2.381 Merkmale. Dabei werden für jede Anfrage sämtliche 375 Kandidatenprofile bewertet. Die konkreten Größen

der Trainings-, Validierungs- und Testmatrizen ergeben sich erst nach der Datensplit-Erzeugung und werden in Kapitel **??** beschrieben.

5.7 Erzeugung der Relevanzlabels

Für jedes Anfrage-Kandidaten-Paar wird ein Relevanzlabel erzeugt, das angibt, ob der Kandidat für die jeweilige Anfrage relevant ist oder nicht (Pipeline step 931 2026). Im verwendeten Trainingsmodus existiert pro Anfrage genau ein korrektes EPLAN-Fenstermakro (EPLAN GmbH & Co. KG 2025). Die Funktion für die Relevanz lautet:

$$y(q, c) = \begin{cases} 1, & \text{wenn } c \text{ das Zielmakro von } q \text{ ist} \\ 0, & \text{sonst} \end{cases} \tag{5.5}$$

Dabei bezeichnet $y(q, c)$ das Relevanzlabel, q die Anfrage und c den Makrokandidaten. Jede Anfragegruppe enthält genau einen positiven Kandidaten und mehrere negative Kandidaten. Die negativen Kandidaten sind dazu da, das Modell darauf zu trainieren, irrelevante Makros korrekt zu erkennen und von relevanten zu unterscheiden. Dadurch werden nicht einzelne Objekte klassifiziert, sondern die korrekte Sortierung der Menge an Kandidaten (Pipeline step 931 2026).

Die Referenzkonfiguration A08 verwendet diese binäre Labeldefinition. Zusätzlich wurde im Rahmen von Konfigurationsexperimenten eine hierarchische Labeldefinition untersucht. Dabei werden Makros derselben Familie sowie Makros mit identischem Familienpräfix durch eine abgestufte Relevanz berücksichtigt. Diese Variante wird in der späteren Gegenüberstellung der Konfigurationsexperimenten als separate Konfiguration betrachtet.

5.8 Erstellung des Trainingsformats

Die berechneten Merkmale werden als Matrix gespeichert. Jede Zeile steht für ein Anfrage-Kandidat-Paar (Pipeline step 931 2026):

$$(q, c_i) \rightarrow [x_1, x_2, \ldots, x_n] \tag{5.6}$$

Dabei bezeichnet q die Anfrage, c_i den Makrokandidaten und $x_1, \ldots, x_n$ die berechneten Features. Zusätzlich wird die Zuordnung jeder Anfrage zu einer Gruppe von Kandidaten gespeichert:

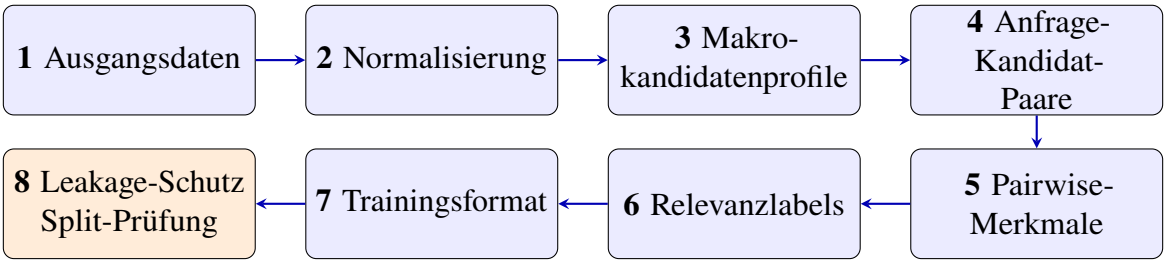


Abbildung 5.1: Verarbeitungsschritte von den Ausgangsdaten bis zum Trainingsformat.

$$G_q = \{(q, c_1), (q, c_2), \ldots, (q, c_n)\} \tag{5.7}$$

Dabei beschreibt G_q die Menge aller Anfrage-Kandidat-Paare für die Anfrage q. Die Informationen aus den Gruppen werden zusammen mit den Featurevektoren an den LambdaRANK-Algorithmus übergeben (Pipeline step 931 2026). Dadurch können die Kandidaten innerhalb derselben Anfrage gegeneinander bewertet werden und somit die optimale Rangfolge erlernt werden.

Für die Referenzkonfiguration A08 entsteht dafür eine Trainingsmatrix mit einer Dimension von 369.000 × 2.381. Die Validierungsmatrix besitzt eine Dimension von 120.750 × 2.381 und die Testmatrix eine Dimension von 94.875 × 2.381. Die Anzahl der Zeilen ergibt sich aus der vollständigen Bewertung aller 375 Kandidatenprofile für jede Anfrage.

5.9 Vermeidung von Zielinformations-Leakage

Beim Aufbau der Trainingsdaten muss sichergestellt werden, dass nur solche Informationen verwendet werden, die auch während einer realen Anfrage verfügbar wären. Die Query-Darstellung wird deshalb auf Merkmale beschränkt, die aus einem eingehenden EPLAN-Artikel oder einem technischen Datenblatt abgeleitet werden können. Direkte Zielinformationen wie target_macro, target_family sowie weitere makrobezogene Zielfelder werden ausgeschlossen. Makrostrukturinformationen werden ausschließlich innerhalb der Kandidatenprofile gespeichert. Zusätzlich werden Kandidatenprofile ausschließlich aus Artikeln des Trainingssplits erstellt. Validierungs- und Testdaten fließen nicht in die Profilerstellung ein. Diese Maßnahmen bilden die Grundlage für die Leakage-Sicherheit der Trainingsdaten. Dieser Schutzmechanismuen werden im Rahmen der Evaluationssuite in Kapitel **??** beschrieben.

Für eine bessere Veranschaulichung des Datenflusses der Datenaufbereitung wird in Abbildung **??** eine schematische Darstellung gezeigt.

6 Datensplit und Training

6.1 Datensplit und Trainingsbasis

Nach der Datenaufbereitung, der Qualitätsprüfung der Daten sowie der Erstellung des idealen Datensatzes erfolgt die Aufteilung in Trainings-, Validierungs- und Testdaten (Pipeline step 911 2026; Pipeline step 913 2026; Ideal dataset 2026; Pipeline step 921 2026). Ziel ist es eine klare Trennung zwischen Modelltraining, Hyperparameteroptimierung und Modellbewertung zu schaffen.

$$\text{Train} = 60\,\%, \qquad \text{Validation} = 20\,\%, \qquad \text{Test} = 20\,\% \tag{6.1}$$

Um reproduzierbare Ergebnisse mit einem erstellten Datensplit zu gewährleisten, wird ein fester Zufallsseed verwendet (Pipeline step 921 2026).

$$\text{RANDOM_SEED} = 42 \tag{6.2}$$

Dabei wird keine reine Zufallsaufteilung vorgenommen, sondern fachliche Zusammenhänge der Daten berücksichtigt, um eine konsistente Verteilung über die Anfragegruppen sicherzustellen. Darüber hinaus wird sichergestellt, dass jedes Zielmakro, welches im Validierungs- oder Testsplit vorkommt, mindestens einmal als Beispiel im Trainingssplit repräsentiert wird. Zielmakros, die im Gesamtdatensatz nur einmal auftreten, werden vollständig dem Trainingssplit zugeordnet. Dies verhindert, dass eng verwandte Beispiele in unterschiedlichen Datensplits landen und somit die Glaubwürdigkeit der Modellbewertung negativ beeinflussen.

Abbildung **??** fasst diese Aufteilung schematisch zusammen.

Für den in dieser Arbeit verwendeten Referenzlauf ergibt sich folgende Aufteilung:

- Training: 984 Artikel
- Validation: 322 Artikel
- Test: 253 Artikel

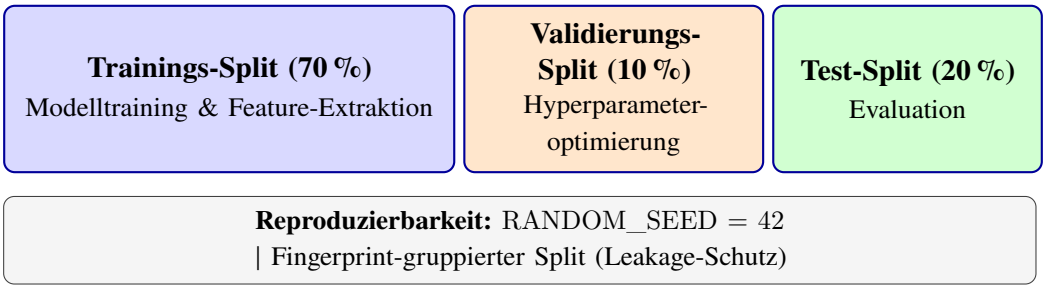


Abbildung 6.1: Schematische Aufteilung der Trainings-, Validierungs- und Testdatenbasis.

6.2 Aufbau der Trainingsmatrix

Nach der Erstellung der Kandidatenprofile werden für jede Anfrage die verfügbarenMakrokandidaten bewertet. Für jede Anfrage werden Anfrage-Kandidat-Paare erzeugt und anhand der Pairwise-Merkmale aus Kapitel 5 beschrieben. Die Referenzkonfiguration A08 verwendet insgesamt 375 Kandidatenprofile und 2.381 Merkmale. Für jede Anfrage werden alle 375 Kandidatenprofile vollständig bewertet.

Dadurch entstehen folgenden Matrizen:

- Trainingsmatrix: 369.000 × 2.381
- Validierungsmatrix: 120.750 × 2.381
- Testmatrix: 94.875 × 2.381

Die Merkmalsmatrizen werden als Sparse-Matrizen gespeichert und mit numerischen Werten des Typs $\mathrm{float32}$ verarbeitet.

6.3 Training des Learning-to-Rank-Modells

Das zentrale Element der Architektur ist das Learning-to-Rank-Modell. Hierfür wird der $\mathrm{LGBMRanker}$ der LightGBM-Bibliothek verwendet (Pipeline step 931 2026). Dieser basiert auf dem in Kapitel 2 beschriebenen LambdaRank-/LambdaMART-Prinzip.

Das Modell wird mit den Parametern

$$\text{objective} = \text{lambdarank} \qquad \text{und} \qquad \text{metric} = \text{ndcg} \tag{6.3}$$

trainiert (Pipeline step 931 2026).

Die für die spätere Referenzkonfiguration A08 ausgewählten Modellparameter lauten:

$$n_estimators = 900 \tag{6.4}$$
$$learning_rate = 0.025 \tag{6.5}$$
$$num_leaves = 15 \tag{6.6}$$
$$min_child_samples = 50 \tag{6.7}$$
$$lambdarank_truncation_level = 5 \tag{6.8}$$

Zusätzlich wird eine Kandidatenprofilbreite von acht Artikeln pro Makro verwendet.

Wie in Kapitel 5 beschrieben, werden Kandidaten einer Anfrage als gemeinsame Rankinggruppe betrachtet. Wiederholend entsteht für jede Anfrage eine Gruppe:

$$G_q = \{(q, c_1), (q, c_2), \ldots, (q, c_n)\} \tag{6.9}$$

mit der Anfrage q und den zugehörigen Makrokandidaten c_i.

Dadurch erlernt das Modell nicht die allgemeine Zugehörigkeit eines einzelnen Kandidaten zu einer Klasse. Stattdessen wird die optimale Kandidatenreihenfolge innerhalb derselben Anfrage gelernt. Ziel dabei ist es, dass der relevanteste Kandidat möglichst weit oben in der Rangliste erscheint.

Die Eingaben für das Modell sind die im Kapitel 5 beschriebenen Pairwise-Merkmale. Das Training erfolgt dabei ausschließlich auf den Trainingsdaten, während die Optimierung der Hyperparameter über die Validierungsdaten erfolgt (Pipeline step 931 2026).

6.4 Hyperparameteroptimierung

Für die Hyperparameteroptimierung wird eine Grid-Search mit insgesamt 15 definierten Parameterkonfigurationen verwendet. Variiert werden dabei folgende Parameter:

- n_estimators,
- learning_rate,
- num_leaves,
- min_child_samples,
- lambdarank_truncation_level.

Die Bewertung dieser Parameterkonfigurationen erfolgt ausschließlich auf dem Validierungssplit. Der Testsplit wird erst nach Abschluss der Modellauswahl für die finale Evaluation verwendet.

Die folgende Tabelle **??** zeigt die untersuchten Hyperparameterkombinationen:

Tabelle 6.1: Variierte Hyperparameter der Grid-Search (Pipeline step 931 2026)

Parameter	**Wertebereich**
n_estimators	300 bis 900
learning_rate	0.025 bis 0.05
num_leaves	15 bis 127
min_child_samples	5 bis 80
lambdarank_truncation_level	5 bis 20

Dabei sind die Hyperparameter wie folgt zu verstehen:

- n_estimators**:** Anzahl der aufgebauten Entscheidungsbäume im Modell. Mehr Bäume = komplexere Zusammenhänge, aber mehr Rechenaufwand.
- learning_rate**:** Lernrate pro Trainingsschritt. Kleinere Werte führen zu vorsichtigerem, aber stabileren Lernen.
- num_leaves**:** Maximale Anzahl der Blätter pro Baum. Mehr Blätter = komplexere Bäume, aber höheres Risiko für Overfitting.
- min_child_samples**:** Minimale Anzahl an Trainingsbeispielen in einem Blatt. Kleinere Werte = mehr Detailtreue pro Blatt, größere Werte = bessere Regularisierung (verhindert Overfitting).
- lambdarank_truncation_level**:** Begrenzung des Ranglistenbereichs, der durch LambdaRank optimiert werden soll (Top 10, Top 20).

Für jede Parameterkombination wird ein vollständiger Trainingsdurchlauf durchgeführt. Die Bewertung erfolgt anschließend auf dem Validierungsdatensatz. Das finale Modell wird auf Basis der Rankingqualität anhand der Validierungsdaten bestimmt (Pipeline step 931 2026).

Die verwendeten Bewertungsmetriken werden später in Kapitel **??** ausführlich beschrieben.

6.5 Modellartefakte und Reproduzierbarkeit

Ein wichtiges Ziel dieser Trainingspipeline besteht darin, die Trainingsabläufe reproduzierbar zu gestalten. Deswegen werden nach jedem Trainingslauf alle relevanten Artefakte gespeichert und dokumentiert (Pipeline step 931 2026). Abbildung **??** zeigt die zugehörige Ablagestruktur.

Dadurch lässt sich sicherstellen, dass jeder Trainingsablauf zu einem späteren Zeitpunkt nachvollzogen und reproduziert werden kann (Pipeline step 931 2026).

Ablagestruktur im Laufverzeichnis:

```
run_directory/
|– model/
|– training/
|– evaluation/
|– summary/
|– error_analysis/
\– consolidated/
```

Gespeicherte Modellartefakte:

- das trainierte LightGBM-Modell,
- die verwendeten Hyperparameter,
- die erzielten Validierungsmetriken,
- die verwendeten Featuredefinitionen,
- die Makrokandidatenprofile,
- die Ergebnisse der Hyperparameteroptimierung,
- die ausgewählten Bestparameter,
- die Validierungsmetriken der Grid-Search,
- das Run-Manifest.

Abbildung 6.2: Strukturierte Ablage der Modellartefakte im Laufverzeichnis.

6.6 Pipeline-Orchestrierung

Die Ausführung der gesamten Pipeline erfolgt über eine zentrale Komponente, die die Orchestrierung übernimmt. Dadurch wird sichergestellt, dass alle Verarbeitungsschritte in der richtigen Reihenfolge ausgeführt werden (Pipeline step 936 2026).

Die folgende Reihenfolge der Schritte stellt den Ablauf der Pipeline dar. Diese können auch aus dem Architekturdiagramm in Abbildung 4.1 entnommen werden:

1. Datenaufbereitung
2. Qualitätsfilterung und Bereinigung
3. Datensplit-Erstellung
4. Kandidatenprofilgenerierung und Featurebildung
5. LightGBM-Learning-to-Rank-Training
6. Konsolidierung der Laufartefakte
7. Evaluation

Jeder Verarbeitungsschritt verwendet dabei die Artefakte des vorherigen Schrittes als Eingabe und erzeugt selbst Artefakte für den nächsten Schritt als Ausgabe. Dadurch wird ein konsistenter Datenfluss von den Roh-Daten bis zum trainierten Modell sichergestellt (Pipeline step 936 2026; Training and ranking pipeline 2026).

7 Evaluierungskonzept

7.1 Evaluationsdatensätze

Die Bewertung der Modelleistung erfolgt getrennt auf Trainings-, Validierungs- und Testdatensätzen. Diese Aufteilung wird im Rahmen des zuvor beschriebenen Kapitels **??** vorgenommen.

Der Testdatensatz wird erst nach der finalen Modellauswahl für die abschließenden Evaluierungen verwendet. Dadurch wird verhindert, dass Informationen aus dem Testdatensatz in die Optimierung des Modells einfließen und die Bewertung verfälschen.

Ein wichtiger Aspekt dabei ist die Behandlung von fachlich Zusammenhängenden Daten. Artikel die denselben Fingerprint besitzen, dürfen nur gemeinsam in einem Teil des Splits vorkommen. Ein Fingerprint repräsentiert die Struktur und die wesentlichen Merkmale eines Artikels, sodass Artikel mit demselben Fingerprint als fachlich zusammenhängend betrachtet werden.

Dadurch wird verhindert, dass nahezu ähnliche Beispiele gleichzeitig im Trainings- und Testdatensatz vorkommen.

7.2 Aufbau der Evaluations-Suite

Die Evaluation des Learning-to-Rank-Modells erfolgt automatisiert mithilfe einer Evaluations-Suite (Evaluation suite 2026), welche in dem Modul step_944_analyze_ltr_run.py verankert ist. Damit wertet das Modul die Vorhersagen des trainierten Modells auf dem Validierungs- und Testdatensatz aus und berechnet die definierten Rankingmetriken sowie zusätzliche Fehleranalysen.

Die Ergebnisse dieser Pipeline werden strukturiert in den folgenden Verzeichnissen abgelegt:

Tabelle 7.1: Aktuelle Evaluations-Suite

Nr.	Skript	Zielsetzung
1	step_944_analyze_ltr_run.py	Berechnung der Rankingmetriken, Fehleranalyse und Evaluationsstatistiken

Tabelle 7.2: Bedeutung der verwendeten Rankingmetriken

Metrik	**Bewerteter Qualitätsaspekt**
Target Recall@k	Position des exakten Ziel-Makros (EPLAN GmbH & Co. KG 2025)
Family Recall@k	Erkennung der korrekten Makrofamilie
MRR	Durchschnittlicher Zielrang
Hierarchical nDCG	Qualität der gesamten Rangfolge

- evaluation/step_941_leakage_safety für die Prüfung auf Zielinformations-Leakage
- evaluation/step_942_retrieval_validation für die Bewertung des Retrieval-Index
- evaluation/step_943_candidate_coverage für die Analyse der Kandidatenabdeckung
- evaluation/step_944_ltr_run_analysis für die Analyse des Learning-to-Rank-Modells

7.3 Verwendete Rankingmetriken

Die theoretischen Grundlagen der verwendeten Rankingmetriken wurden bereits in Kapitel 2 behandelt. Für die Bewertung des Systems aus dieser Arbeit werden die folgenden Kennzahlen verwendet:

- Target Recall@1, @3, @5 und @10
- Family Recall@1, @3, @5 und @10
- Mean Reciprocal Rank (MRR)
- Hierarchical nDCG@1, @3, @5 und @10
- Rangstatistiken der Ziel-Makros (EPLAN GmbH & Co. KG 2025)

Durch diese Metriken können die Zielmakroerkennung, der Familienerkennung sowie die allgemeinen Rankingqualität bewertet werden.

7.3.1 Bedeutung der Metriken innerhalb der Evaluation

Die Bewertungsmetriken bilden gemeinsam verschiedene Aspekte der Rankingqualität ab, wie in Tabelle **??** zusammengefasst.

7.4 Leakage-Sicherheitsprüfung

Ein zentraler Bestandteil in der Evaluation ist eine Leakage-Sicherheitsprüfung auf Zielinformationen. Unter Leakage wird die unbeabsichtigte Nutzung von Informationen verstanden, die zum Zeitpunkt einer realen Vorhersage nicht verfügbar sein dürften und dadurch zu unrealistisch guten

Modellergebnissen führen würden (Kaufman u. a. 2012). Da dieses System auf Artikel (EPLAN GmbH & Co. KG 2026b) - Makro (EPLAN GmbH & Co. KG 2025) -Zuordnungen trainiert wird, besteht das Risiko, das Informationen über das tatsächliche Ziel-Makro (EPLAN GmbH & Co. KG 2025) in die Trainings- und Evaluierungsdaten gelangen können und somit die Aussagekraft der verwendeten Evaluationsmetriken verfälscht wird.

Um dies zu vermeiden, enthält die Pipeline eine Leakage-Prüfung, die vor der eigentlichen Modellbewertung ausgeführt wird. Die Implementierung erfolgt in `step_941_validate_leakage_safety.py` (Pipeline step 941 2026).

7.4.1 Ziele der Leakage-Prüfung

Die Ziele der Leakage-Prüfung sind:

- Identifikation verbotener Zielinformationen innerhalb der Query-Daten
- Erkennung direkter Ziel-Makro (EPLAN GmbH & Co. KG 2025) -Referenzen
- Prüfung auf Familien- und Zielinformationen innerhalb von Eingangsmerkmalen
- Kontrolle der Kandidatenprofile
- Überprüfung der Datensplits auf Überschneidungen
- Sicherstellung der Trennung von Training, Validierung und Test

Somit wird sichergestellt, dass das Modelltraining nur mit den Informationen erfolgt, die später auch im Produktivbetrieb verfügbar sind.

7.4.2 Prüfung der Query-Daten

Die verwendete Query-Darstellung repräsentiert einen eingehenden Artikel (EPLAN GmbH & Co. KG 2026b) beziehungsweise die Informationen aus einem Datenblatt, die für eine Suchanfrage an das trainierte Modell wichtig sind.

Wichtig hierbei ist es, dass keine direkten Hinweise auf das Ziel-Makro (EPLAN GmbH & Co. KG 2025) in den Query-Daten enthalten sind, welche unmittelbar auf dieses verweisen können.

Deswegen untersucht die Leakage-Prüfung folgenden Punkte:

- Ziel-Makro (EPLAN GmbH & Co. KG 2025) -Kennungen
- Zielfamilieninformationen
- direkten Ziel-Makro (EPLAN GmbH & Co. KG 2025) -Tokens
- verbotenen Metadatenfeldern
- impliziten Zielhinweisen in den Featurevektoren

Sobald solche Informationen gefunden werden, wird der Lauf als Ungültig markiert.

7.4.3 Prüfung der Datensplits

Neben der Query-Prüfung erfolgt auch eine Prüfung des erzeugten Datensplits. Dabei wird überprüft, ob unerlaubte Überschneidungen zwischen Training, Validierung und Test vorhanden sind.

Dabei werden folgende Aspekte überprüft:

- Fingerprint-Gruppen
- Fenstermakrozuordnungen (EPLAN GmbH & Co. KG 2025)
- Kandidatenprofile
- technische Zusammenhangskomponenten
- gruppierte Artikelstrukturen

Das Ziel dieser Prüfung ist es, sicherzustellen, dass keine fachlich identischen Beispiele gleichzeitig in mehreren Datensplits vorhanden sind. Dadurch wird ein künstliche Modell-Verbesserung verhindert, die die Generalisierungsfähigkeit des Modells beeinträchtigen würde.

7.4.4 Prüfung der Kandidatenprofile

Ein weiterer Bereich der Prüfung betrifft die Makro-Kandidatenprofile (EPLAN GmbH & Co. KG 2025). Da diese Profile aus den Trainingsdaten entstehen, muss sichergestellt werden, dass keine Informationen aus den Validierungs- oder Testdaten in die Kandidatenprofile einfließen.

Deswegen werden sämtliche Profilinformationen auf ihre Herkunft geprüft, um sicherzustellen, dass ausschließlich Daten des Trainingssplits zur Erstellung der Kandidatenprofile verwendet werden (Retrieval leakage guard 2026).

7.4.5 Evaluierungsergebnis

Nach der Leakage-Prüfung wird ein struktruierter Sicherheitsbericht erstellt. Der Evaluationslauf gilt nur dann als bestanden, wenn keine Verstöße gegen die definierten Leakage-Regeln festgestellt wurden. Diese Basis bildet die Voraussetzung für die Interpretierbarkeit der nachfolgenden Rankingmetriken.

Für den verwendeten Datensatz wurden keine Fingerprint-Konflikte noch Fingerprint-Leaks festgestellt. Es sind auch keine Ziel-Makros (EPLAN GmbH & Co. KG 2025) oder Makrofamilien in Validierungs- beziehungsweise Testdaten vorhanden, ohne zuvor im Trainingssplit vertreten zu sein.

8 Ergebnisse und Diskussion

8.1 Ergebnisse des Validierungssplits

Die Ergebnisse des Validierungssplits werden im späteren Verlauf für die Auswahl der besten Modellkonfiguration verwendet.

Tabelle **??** fasst die wichtigsten Rankingmetriken des Referenzlaufs A08 auf dem Validierungssplit zusammen.

Die Ergebnisse zeigen, dass das Modell bereits nach dem Test durch den Validierungssplit die korrekte Makrofamilie besser identifiziert als die exakte Ziel-Makro (EPLAN GmbH & Co. KG 2025). Während das korrekte Ziel-Makro (EPLAN GmbH & Co. KG 2025) in 31,37 % der Anfragen auf Platz 1 steht, wird die korrekte Makrofamilie in 81,99 % der Anfragen auf die erste Position gesetzt.

Durch den Target Recall@10 lässt sich erkennen, dass das Modell in 62,11 % der Anfragen das korrekte Ziel-Makro (EPLAN GmbH & Co. KG 2025) innerhalb der Top-10-Vorschläge platziert. Intessant ist der Abstand zwischen Family Recall@10 (89,13 %) und Target Recall@10 (62,11 %). Dies deutet darauf hin, dass viele Fehlkategorisierungen innerhalb derselben Makrofamilie stattfinden.

Die erzielten Ergebnisse bei MRR von 0,4121 % sowie ein hierarchisches nDCG@10 von 0,6766 % zeigen zusätzlich, dass beim Validierungssplit das Modell in der Lage ist eine sinnvolle Reihen-

Tabelle 8.1: Ergebnisse des Referenzlaufs A08 auf dem Validierungssplit

Metrik	**Wert**
Target Recall@1	0,3137
Target Recall@3	0,4472
Target Recall@5	0,5031
Target Recall@10	0,6211
Family Recall@1	0,8199
Family Recall@10	0,8913
MRR	0,4121
Hierarchical nDCG@10	0,6766

Tabelle 8.2: Ergebnisse des Referenzlaufs A08 auf dem Testsplit

Metrik	**Wert**
Target Recall@1	0,3478
Target Recall@3	0,4783
Target Recall@5	0,5494
Target Recall@10	0,6798
Family Recall@1	0,8735
Family Recall@10	0,9368
MRR	0,4534
Hierarchical nDCG@10	0,7449

folge zu erzeugen, es jedoch weiterhin Schwierigkeiten bei der korrekten Auflösung technischer Varianten besitzt.

8.2 Ergebnisse des Testsplits

Nach dem die Modellauswahl abgeschlossen wurde, erfolgt die finale Evaluation des ausgewählten Modells auf dem unberührten Testsplit.

Tabelle **??** zeigt die erzielten Ergebnisse des Referenzlaufs A08 des Testsplits.

Beim Testsplit zeigt sich ein ähnliches Bild wie beim Validierungssplit. Das Modell sagt bei 34,78 % der Anfragen das korrekte Ziel-Makro (EPLAN GmbH & Co. KG 2025) auf Platz 1 voraus, während die korrekte Makrofamilie in 87,35 % der Anfragen auf die erste Position gesetzt wird.

Wie beim Validierungsplit ist auch hier der deutliche Abstand zwischen Family Recall@10 (93,68 %) und Target Recall@10 (67,98 %) zu erkennen.

Der erreichte MRR- Wert von 45,34 % sowie die hierarchische nDCG@10 von 74,49 % bestätigen zusätzlich die Erkenntnisse aus dem Validierungssplit.

8.3 Fehleranalyse

Um die Modelleistung detailliert erfassen zu können, werden die Top-1-Ergebnisse des Testsplits analysiert. Dabei werden die fachlichen Beziehungen zum tatsächlichen Zielmakro untersucht.

Tabelle **??** zeigt die Verteilung der einzelnen Fehlerkategorien.

Tabelle 8.3: Top-1-Analyse der Vorhersagen auf dem Testdatensatz

Kategorie	**Anzahl**	**Anteil**
Exaktes Ziel-Makro (EPLAN GmbH & Co. KG 2025)	88	34,78 %
Gleiche Familie, falsches Target	133	52,57 %
Gleicher Familienpräfix	12	4,74 %
Andere Familie	20	7,91 %

Tabelle 8.4: Rangstatistische Kennzahlen des Testsplits

Kennzahl	**Wert**
Erwarteter Zielrang (Mean Rank)	23,96
Median Rank	4
90 %-Perzentil Rank	61,6
Maximaler Rang	360

Die Fehleranlyse bestätigt die Beobachtungen aus den Metriken. Von insgesamt 253 Testanfragen werden 133 Fälle einer gleichen Familie, aber einer falschen Variante zugeordnet, was 52,57 % entspricht. In 7,91 % der Fälle wird eine andere Familie vorhergesagt. Das zeigt, dass das Modell die grundlegenden technischen Zusammenhänge zwischen Artikeln und Makrofamilien zuverlässig erlernt hat. Die Analyse zeigt, dass die zentrale Herausforderung die Identifikation des korrekten Ziel-Makros (EPLAN GmbH & Co. KG 2025) innerhalb der richtigen Makrofamilie ist. Dafür muss die Variantenauflösung innerhalb der Makrofamilie weiter verbessert werden. Die große Differenz zwischen dem Target Recall@1 und dem Family Recall@1 unterstreicht diese Herausforderung zusätzlich. Dies wird im Kapitel **??** genauer analysiert.

8.4 Rangstatistische Analyse

Eine weitere wichtige Analyse ist die statistische Auswertung der erzeugten Ranglisten.

Die wichtigsten Rangstatistiken des Testdatensatzes werden in Tabelle **??** dargestellt.

Der Median Rank von 4 zeigt, dass das korrekte Makro (EPLAN GmbH & Co. KG 2025) in der Hälfte der Testanfragen unter den ersten vier Positionen der Rangliste erscheint. Der mittlere Rang (Mean Rank) von 23,96 zeigt gleichzeitig, dass es einzelen schwierige Anfrage gibt, die deutliche schlechter eingeordnet werden. Diese Annahme wird durch das 90 %-Perzentil von 61,6 bestätigt. Dabei sitzt das Makro (EPLAN GmbH & Co. KG 2025) in 90 % der Testanfragen unter den ersten 61,6 Positionen der Rangliste.

Auch die Rangstatistiken zeigen, dass das Modell in der Lage ist, die korrekte Makro-Familie in den meisten Fällen weit oben in der Rangliste zu platzieren.

Tabelle 8.5: Vergleich von regulärer Trainingsbewertung, Leave-One-Out-Analyse, Validation und Test

Metrik	**Train (LOO)**	**Validation**	**Test**
Recall@1	0,1870	0,3137	0,3478
Recall@3	0,3028	0,4472	0,4783
Recall@10	0,4695	0,6211	0,6798
MRR	0,2864	0,4121	0,4534
Family Recall@1	0,7246	0,8199	0,8735
Hierarchical nDCG@10	0,5945	0,6766	0,7449
Mean Rank	25,93	23,96	23,96
Median Rank	13,0	4,0	4,0
90 %-Perzentil	62,0	61,6	61,6

8.5 Analyse der Generalisierungsfähigkeit

Zur Beurteilung der Generalisierungsfähigkeit des Modells wurde eine zusätzliche Leave-One-Out Kontrollanalyse durchgeführt. Um eine Verfälschung der Ergebnisse der Trainigsanalyse zu verhindern, werden bei einer Anfrage genau dieser Artikel aus der Anfragemenge ausgeschlossen.

Tabelle **??** stellt die Ergebnisse der Leave-One-Out-Analyse (LOO) sowie der Validierungs- und Testergebnisse gegenüber.

Es zeigt sich, dass die Leave-One-Out-Bewertung eine strengere Evaluationsmethode als die Bewertung durch den Validierungs- und Testdatensatz darstellt. Die Ursache dafür liegt im Support der Kandidatenprofile. Von den vorhanden 375 Kandidatenprofilen basieren insgesamt 302 Profile ausschließlich auf einem Traingsartikel. Wenn dieser Artikel aus der Anfragemenge ausgeschlossen wird, enthält das entsprechende Kandidatenprofil keine unterstützenden Trainingsdaten mehr und kann dadruch keinen Beitrag zur korrekten Vorhersage leisten.

Bei den Validierungs- und Testdaten tritt dieser Effekt nicht auf, da durch die im Datensplitt erzwungene Trainigsabdeckung jedes Ziel-Makro (EPLAN GmbH & Co. KG 2025) mindestens einen Artikel in jedem Split enthält, welcher nicht identisch zu den andern beiden ist.

8.6 Vergleich der Konfigurationsvarianten

Um verschiedene Modellkonfigurationen zu vergleichen, wurde die aktuell Versuchsserie A01 bis A09 durchgeführt. Dabei wurden unterschiedliche Strategien, Profielbreiten und Regularisierungsparameter getestet. Ziel davon ist es, die Einflüsse einzelner Parameter auf die Modellleistung zu analysieren.

Tabelle 8.6: Untersuchte Konfigurationsparameter

Parameter	**Bedeutung**
Same-Family-Gewicht	Negative Kandidaten aus derselben Makrofamilie werden innerhalb der Verlustfunktion unterschiedlich gewichtet. Dadurch kann die Variantenauflösung stärker betont werden.
Profilbreite	Begrenzt die Anzahl der gespeicherten Profilwerte innerhalb eines Kandidatenprofils. Kleinere Profilbreiten erzeugen eine kompaktere Kandidatenrepräsentation.
L1-Regularisierung (α)	Bestraft große Modellgewichte und reduziert die Modellkomplexität. Dadurch werden irrelevante Merkmale stärker unterdrückt.
L2-Regularisierung (λ)	Stabilisiert die Gewichtung der einzelnen Merkmale und reduziert die Gefahr einer Überanpassung an einzelne Trainingsbeispiele.
Min. Split Gain	Definiert den minimal notwendigen Informationsgewinn um eine Baumaufteilung im LightGBM-Modell zu ermöglichen. Höhere Werte führen dadurch zu konservativeren Entscheidungsbäumen.

8.6.1 Untersuchte Konfigurationsparameter

Die verschiedenen Varianten der Konfiguration haben unterschiedliche Auswirkungen auf die Trainingslabels, die Kandidatenprofile sowie der Modellregularisierung. Die untersuchten Konfigurationsparameter werden in folgender Tabelle **??** erläutert.

A02 bis A07 wurden so gewählt, dass immer nur ein Parameter gezielt verändert wurde. Dadurch lassen sich die Auswirkungen der einzelnen Konfigurationsparameter isoliert untersuche. Bei A08 und A09 werden mehrere Parameter (L1 und L2) gleichzeitig verändert, da diese bei LightGBM konzeptionell derselbe Hebel sind (Bestrafung der Blattgewichte). Um eine sinnvolle ßtärkere Regularisierungßu erzielen, ist es in der Praxis sinnvoll, beide gemeinsam hochzudrehen, nicht nur einen isoliert. Bei A09 kommt zusätzlich ein positiver Wert für $\mathrm{min_split_gain}$ zum Einsatz um dadurch einen noch größeren Einfluss auf eine ßtärkere Regularisierungßu erzielen.

8.6.2 Referenzkonfiguration A01

A01 dient als Referenzkonfiguration und bildet die Basis für die Vergleichsexperimente. Sie enthält die Standardwerte für alle untersuchten Konfigurationsparameter. Alle weiteren Konfigurationen A02 bis A09 werden gegenüber dieser bewertet.

Die Referenzparameter lauten:

Ausgehen von dieser Basis werden die folgenden Varianten der Konfigurationen A02 bis A09 verändert.

Tabelle **??** zeigt die gegenüber A01 vorgenommenen Veränderrungen.

Tabelle 8.7: Referenzkonfiguration A01

Parameter	**Wert**
Exaktes Targettraining	Ja
Same-Family-Gewicht	5,0
Profilbreite	16
L1-Regularisierung (α)	0,05
L2-Regularisierung (λ)	0,2
Min. Split Gain	0,0

Tabelle 8.8: Abweichungen der Konfigurationen gegenüber der Referenz A01

Konfiguration	**Änderung gegenüber A01**
A02	Same-Family-Gewicht von 5,0 auf 2,0 reduziert
A03	Same-Family-Gewicht von 5,0 auf 8,0 erhöht
A04	Same-Family-Gewicht von 5,0 auf 12,0 erhöht
A05	Hierarchische Relevanzlabels statt ausschließlich exakter Zielmakros
A06	Profilbreite von 16 auf 12 reduziert
A07	Profilbreite von 16 auf 8 reduziert
A08	L1-Regularisierung auf 0,1 und L2-Regularisierung auf 0,5 erhöht
A09	L1-Regularisierung auf 0,5, L2-Regularisierung auf 1,0 und $\mathrm{min_split_gain}$ auf 0,01 erhöht

Dadurch lässt sich der Einfluss der jeweiligen Parameteränderung der entsprechenden Kofiguration zuordnen.

8.6.3 Ergebnisse der Vergleichskonfigurationen

Tabelle **??** zeigt die getesteten Modellkonfigurationen und deren Leistung im Vergleich.

8.6.4 Einfluss des Same-Family-Gewichts

Mit den Konfigurationen A02 bis A04 wird der Einfluss des Same-Family-Gewichts untersucht. Die Ergebnisse zeigen, dass eine Verringerung des Gewichts auf 2 den Recall@1 auf 31,62 % senkt, während eine Erhöhung auf 8 keine Auswirkung zeigt. Die Gewichtung auf 12 führt zu einer deutlichen Verschlechterung des Recall@1 auf 29,25 %.

Tabelle 8.9: Ergebnisse der A01–A09-Konfigurationen auf dem Testdatensatz

Konfiguration	**Recall@1**	**Recall@3**	**Recall@10**	**MRR**
A01 Referenz	0,3241	0,4822	0,6798	0,4418
A02 Gewicht 2	0,3162	0,4862	0,6680	0,4399
A03 Gewicht 8	0,3241	0,4743	0,6838	0,4397
A04 Gewicht 12	0,2925	0,4783	0,6759	0,4195
A05 Hierarchisch	0,3162	0,4783	0,6719	0,4328
A06 Profilbreite 12	0,3162	0,4862	0,6877	0,4374
A07 Profilbreite 8	0,3241	0,4862	0,6719	0,4439
A08 Regularisierung mittel	**0,3478**	0,4783	0,6798	**0,4534**
A09 Regularisierung stark	0,3399	0,4783	**0,6917**	0,4505

Daraus lässt sich schließen, dass ein zu hohes Same-Family-Gewicht die Lernstabilität negativ beeinflusst.

8.6.5 Einfluss der hierarchischen Labeldefinition

A05 untersucht die hierarchische Relevanzdefinition. Makros (EPLAN GmbH & Co. KG 2025) derselben Familie und desselben Präfixes werden dabei hierarchisch relevant betrachtet. Es zeigt sich jedoch keine Verbesserung gegen dem exakten Targettraining. Der Recall@1 sinkt von 32,41 % auf 31,62 % und die MRR von 44,18 % auf 43,28 %. Die dadurch entstandene reichaltigere Zieldefinintion hat somit keinen positiven Einfluss auf die Modellleistung.

8.6.6 Einfluss der Profilbreite

In den Konfigurationen A06 und A07 werden unterschiedliche Profilbreiten untersucht. Bei A06 beträgt diese zwölf bei A07 acht. Die Erhöhung der Profilbreite führt zu einer Verschlechterung des Recall@1 von 32,41 % auf 31,62 % und der MRR von 44,18 % auf 43,74 %. Eine Verringerung der Profilbreite führt zu keiner nennenswerten Veränderung der Modellleistung. Die Ergebnisse zeigen, dass sich eine verringerte Profilbreite die Modelleistung nicht signifikant verbessert.

8.6.7 Einfluss der Regularisierung

Die deutlichsten Verbesserungen werden durch die Veränderung der Regularisierung erzielt. A08 verwendet eine mittlere L1- und L2-Regularisierung ($\alpha = 0{,}1$, $\lambda = 0{,}5$). Dadurch wird mit einem Recall@1 von 0,3478 der höchste Top-1-Wert aller untersuchten Konfigurationen erreicht. Auch der höchste MRR Wert von 0,4534 wird durch A08 erreicht. Mit A09 wird die Regularisierung nochmals deutlich erhöt ($\alpha = 0{,}5$, $\lambda = 1{,}0$). Zusätzlich wird ein positiver Wert für

Tabelle 8.10: Ergebnisse der besten Konfiguration A08

Kennzahl	**Wert**
Recall@1	0,3478
Family Recall@1	0,8735
Recall@10	0,6798
Family Recall@10	0,9368
MRR	0,4534
Mittlerer Zielrang	23,96
90 %-Perzentil	61,6
Schlechtester Rang	360

min_split_gain gesetzt, um die Stabilität der Baumaufteilung zu erhöhen. Diese Konfiguration erreicht den höchsten Recall@10 von 0,6917 und die zweithöchste MRR von 0,4505.

Daraus lässt sich ableiten, dass eine stärkere Regularisierung die Modellgeneralisierung verbessert. Dies ist vor dem Hintergrund der sogenannten Long-Tail-Verteilung der Makros (EPLAN GmbH & Co. KG 2025) plausibel, da 56,8 % aller Zielmakros ein Trainingsbeispiel und 74,4 % weniger als drei Trainingsbeispiele besitzen.

8.6.8 Beste Konfiguration der Studie

Die Beste Konfiguration der Studie ist A08, da sie sowohl den höchsten Recall@1 als auch den höchsten MRR erreicht.

Die wichtigsten Kennzahlen dieser Variante sind in Tabelle **??** dargestellt.

Die Fehleranlyse dieser Konfiguration zeigt, dass 7,9 % aller Anfragen einer fachlich falschen Makro-Familie zugeordnet wurden. 52,6 % der Fehlklassifikationen erfolgen innerhalb der korrekten Makrofamilie. Die größte Herausforderung liegt damit weiterhin in der Variantenauflösung technisch ähnlicher Makrofamilien.

9 Diskussion, Grenzen und Ausblick

10 Zusammenfassung und Fazit

Erklärung zur Abschlussarbeit

Hiermit versichere ich, die eingereichte Abschlussarbeit selbständig verfasst und keine andere als die von mir angegebenen Quellen und Hilfsmittel benutzt zu haben. Wörtlich oder inhaltlich verwendete Quellen wurden entsprechend den anerkannten Regeln wissenschaftlichen Arbeitens zitiert. Ich erkläre weiterhin, dass die vorliegende Arbeit noch nicht anderweitig als Abschlussarbeit eingereicht wurde. Das Merkblatt zum Täuschungsverbot im Prüfungsverfahren der Hochschule Augsburg habe ich gelesen und zur Kenntnis genommen. Ich versichere, dass die von mir abgegebene Arbeit keinerlei Plagiate, Texte oder Bilder umfasst, die durch von mir beauftragte Dritte erstellt wurden.

Augsburg, den 7. September 2026

Andreas Alstetter

Abbildungsverzeichnis

1.1 Grundidee der automatisierten Zuordnung von EPLAN-Artikeln zu EPLAN-Fenstermakros. 3

2.1 Grundprinzip eines Learning-to-Rank-Systems zur Makroempfehlung. 4

3.1 Verzahnung von DSRM und ADR. Die DSRM bildet den übergeordneten Forschungsrahmen, während ADR als iterativer Entwicklungszyklus innerhalb der Phase „Design & Entwicklung des Artefakts“ eingesetzt wird. 13

4.1 End-to-End-Architektur der retrievalfreien Learning-to-Rank-Pipeline. 15

5.1 Verarbeitungsschritte von den Ausgangsdaten bis zum Trainingsformat. 22

6.1 Schematische Aufteilung der Trainings-, Validierungs- und Testdatenbasis. 24
6.2 Strukturierte Ablage der Modellartefakte im Laufverzeichnis. 27

Tabellenverzeichnis

5.1 Statistische Kennzahlen des verwendeten Datensatzes 16
5.2 Analysierte Merkmalsgruppen der EPLAN-Fenstermakros 17
5.3 Aufbereitete Merkmalsgruppen der EPLAN-Artikeldaten 18
5.4 Gruppen der Pairwise-Merkmale 20

6.1 Variierte Hyperparameter der Grid-Search (Pipeline step 931 2026) 26

7.1 Aktuelle Evaluations-Suite 28
7.2 Bedeutung der verwendeten Rankingmetriken 29

8.1 Ergebnisse des Referenzlaufs A08 auf dem Validierungssplit 32
8.2 Ergebnisse des Referenzlaufs A08 auf dem Testsplit 33
8.3 Top-1-Analyse der Vorhersagen auf dem Testdatensatz 34
8.4 Rangstatistische Kennzahlen des Testsplits 34
8.5 Vergleich von regulärer Trainingsbewertung, Leave-One-Out-Analyse, Validation und Test 35
8.6 Untersuchte Konfigurationsparameter 36
8.7 Referenzkonfiguration A01 37
8.8 Abweichungen der Konfigurationen gegenüber der Referenz A01 37
8.9 Ergebnisse der A01–A09-Konfigurationen auf dem Testdatensatz 38
8.10 Ergebnisse der besten Konfiguration A08 39

Literaturverzeichnis

Baeza-Yates, Ricardo und Berthier Ribeiro-Neto (2011). *Modern Information Retrieval: The Concepts and Technology behind Search*. 2. Aufl. Addison-Wesley.

Bundesministerium für Wirtschaft und Energie (2026). *Wirtschaft und Forschung richten Plattform Industrie 4.0 auf industrielle KI aus*. Bundesministerium für Wirtschaft und Energie (siehe S. 2).

Burges, Christopher J. C. (2010). „From RankNet to LambdaRank to LambdaMART: An Overview“. In: *Microsoft Research Technical Report* (siehe S. 4, 8, 9).

Cormack, Gordon V., Charles L. A. Clarke und Stefan Büttcher (2009). „Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods“. In: *SIGIR 2009*, S. 758–759. DOI: 10.1145/1571941.1572114.

EPLAN GmbH & Co. KG (2025). *Makros*. EPLAN-Plattform Infoportal (siehe S. 2–4, 11, 14, 16–19, 21).

EPLAN GmbH & Co. KG (2026a). *Eplan, Software und Services für die Elektrokonstruktion*. https://www.eplan.com/de-de/. Zugegriffen am 02.09.2026 (siehe S. 17).

EPLAN GmbH & Co. KG (2026b). *Artikelverwaltung*. EPLAN-Hilfe (siehe S. 2–4, 11, 14, 16, 18).

Evaluation suite (2026). *Evaluation suite for Learning-to-Rank macro ranking*. Interner Projektquellcode.

Gao, Luyu, Zhuyun Dai und Jamie Callan (2021). „COIL: Revisit Exact Lexical Match in Information Retrieval with Contextualized Inverted List“. In: *Proceedings of NAACL-HLT 2021*, S. 3030–3042. DOI: 10.18653/v1/2021.naacl-main.241.

Ideal dataset (2026). *Ideal dataset selection from cleaned Current articles*. Interner Projektquellcode.

Järvelin, Kalervo und Jaana Kekäläinen (2002). „Cumulated Gain-Based Evaluation of IR Techniques“. In: *ACM TOIS*. Bd. 20. 4, S. 422–446. DOI: 10.1145/582415.582418 (siehe S. 6, 7).

Joshi, Tripuresh, Bhavesh Joshi, Karimulla Syed, S. Vijayarani, Sandeep Kumar und Naveen Naval (2024). „Industrial data analysis using Machine Learning Techniques“. In: *2024 IEEE North Karnataka Subsection Flagship International Conference (NKCon)*. IEEE. DOI: 10.1109/NKCon62728.2024.10774728 (siehe S. 2).

Kabir, Md. Ahsanul, Mohammad Al Hasan, Aritra Mandal, Daniel Tunkelang und Zhe Wu (2024). „A Survey on E-Commerce Learning to Rank“. In: *Procedia Computer Science* (siehe S. 5).

Kaufman, Shachar, Saharon Rosset und Claudia Perlich (2012). „Leakage in Data Mining: Formulation, Detection, and Avoidance“. In: *ACM Transactions on Knowledge Discovery from Data* 6.4. DOI: 10.1145/2382577.2382579.

Liu, Tie-Yan (2009). „Learning to Rank for Information Retrieval“. In: *Foundations and Trends in Information Retrieval* 3.3, S. 225–331. DOI: 10.1561/1500000016 (siehe S. 4, 5, 8).

Lyzhin, Ivan, Aleksei Ustimenko, Andrey Gulin und Liudmila Prokhorenkova (2023). „Which Tricks Are Important for Learning to Rank?“ In: *Proceedings of ICML* (siehe S. 10).

Manning, Christopher D., Prabhakar Raghavan und Hinrich Schütze (2008). *Introduction to Information Retrieval*. Cambridge University Press (siehe S. 5).

Nayak, Srikumar (2025). „Enhancing Manufacturing with Artificial Intelligence: An Overview of the Present Situation, Insights, Recommended Practices, Obstacles and Future Trends“. In: *World Journal of Advanced Research and Reviews* 28.3, S. 009–019. DOI: 10.30574/wjarr.2025.28.3.4033 (siehe S. 3).

Peffers, Ken, Tuure Tuunanen, Marcus A. Rothenberger und Samir Chatterjee (2007). „A Design Science Research Methodology for Information Systems Research“. In: *Journal of Management Information Systems* 24.3, S. 45–77. DOI: 10.2753/MIS0742-1222240302 (siehe S. 12).

Pfenning, Michael (2022). *Wie Maschinenbauer die Variantenvielfalt beherrschen* (siehe S. 2).

Pipeline step 640 (2026). *Pipeline step 640: Macro candidate retrieval*. Interner Projektquellcode.

Pipeline step 911 (2026). *Pipeline step 911: Raw EPLAN article XML parsing and data preparation*. Interner Projektquellcode (siehe S. 16–18).

Pipeline step 913 (2026). *Pipeline step 913: Training data quality filtering and quarantine*. Interner Projektquellcode.

Pipeline step 921 (2026). *Pipeline step 921: Fingerprint-grouped train, validation, and test split*. Interner Projektquellcode.

Pipeline step 931 (2026). *Pipeline step 931: Learning-to-Rank macro ranker training*. Interner Projektquellcode (siehe S. 18, 20, 21).

Pipeline step 936 (2026). *Pipeline step 936: Full macro-LTR training-run orchestration*. Interner Projektquellcode.

Pipeline step 941 (2026). *Pipeline step 941: Leakage safety validation for evaluation runs*. Interner Projektquellcode.

Pipeline step 942 (2026). *Pipeline step 942: Retrieval index validation*. Interner Projektquellcode.

Pipeline step 943 (2026). *Pipeline step 943: Candidate coverage analysis*. Interner Projektquellcode.

Pipeline step 944 (2026). *Pipeline step 944: Learning-to-Rank run analysis*. Interner Projektquellcode.

Retrieval leakage guard (2026). *Retrieval leakage guard for label-clean query and candidate data*. Interner Projektquellcode.

Robertson, Stephen und Hugo Zaragoza (2009). „The Probabilistic Relevance Framework: BM25 and Beyond“. In: *Foundations and Trends in Information Retrieval* 3.4, S. 333–389. DOI: 10.1561/1500000019.

Sein, Maung K., Ola Henfridsson, Sandeep Purao, Matti Rossi und Rikard Lindgren (2011). „Action Design Research“. In: *MIS Quarterly* 35.1, S. 37–56. DOI: 10.2307/23043488 (siehe S. 12).

Thakur, Nandan, Nils Reimers, Andreas Rücklé, Abhishek Srivastava und Iryna Gurevych (2021). „BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models“. In: *NeurIPS Datasets and Benchmarks*.

Training and ranking pipeline (2026). *Training and ranking pipeline for EPLAN macros*. Interner Projektquellcode und Dokumentation, README.md und docs/PROJECT_DOCUMENTATION.md (siehe S. 14, 16).

Voorhees, Ellen M. und Dawn M. Tice (1999). „The TREC-8 Question Answering Track Evaluation“. In: *Proceedings of the Eighth Text REtrieval Conference (TREC-8)*. DOI: 10.6028/NIST.SP.500-246.qa-overview (siehe S. 6).

Zhang, Yue, Chengcheng Hu, Yuqi Liu, Hui Fang und Jimmy Lin (2021). „Learning to Rank in the Age of Muppets: Effectiveness-Efficiency Tradeoffs in Multi-Stage Ranking“. In: *SustaiNLP 2021*.

Zheng, Kai, Haijun Zhao, Rui Huang, Beichuan Zhang, Na Mou, Yanan Niu, Yang Song, Hongning Wang und Kun Gai (2024). „Full Stage Learning to Rank: A Unified Framework for Multi-Stage Systems“. In: *WWW 2024*.

Zhu, Yutao, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou und Ji-Rong Wen (2024). „Large Language Models for Information Retrieval: A Survey“. In: *arXiv preprint arXiv:2308.07107* (siehe S. 6).